

Phase 4.1: SSA Construction

Overview

Static Single Assignment (SSA) form is an intermediate representation where each variable is assigned exactly once. This property simplifies many compiler optimizations by making def-use chains explicit and eliminating ambiguity about which definition reaches a particular use.

Components

SSA Types (`SsaTypes.hpp/cpp`)

SsaVariable

Represents a versioned variable in SSA form:

- `id` : Unique identifier
- `base_name` : Original variable name (e.g., "x")
- `version` : Version number (e.g., 0, 1, 2)
- `type` : BDI type of the variable
- `defining_block` : Block where this variable is defined

Example: Variable `x` with three assignments becomes `x_0` , `x_1` , `x_2` .

PhiNode

Represents a phi function at control flow merge points:

- `result` : SSA variable produced by this phi
- `block` : Block containing this phi node
- `operands` : Map from predecessor block ID to SSA variable from that path

Example:

```
x_2 = phi(BB1: x_0, BB2: x_1)
```

ControlFlowGraph

Represents the control flow structure:

- Basic blocks with predecessors and successors
- Entry block identification
- Edge management

DominanceInfo

Stores dominance relationships:

- `immediate_dominator` : The immediate dominator (idom)
- `dominance_frontier` : Set of blocks in the dominance frontier
- `dominated_blocks` : Blocks dominated by this block

SSA Construction (`SsaConstruction.hpp/cpp`)

Algorithm Overview

The SSA construction follows the classic Cytron et al. algorithm:

1. Compute Dominance Information

- Calculate immediate dominators using iterative algorithm
- Compute dominance frontiers from immediate dominators

2. Insert Phi Nodes

- For each variable definition, insert phi nodes at dominance frontiers
- Iterate until no new phi nodes are needed

3. Rename Variables

- Traverse dominator tree
- Assign fresh versions to each variable definition
- Update phi node operands

Dominance Computation

Immediate Dominators:

- Entry block dominates itself
- For other blocks, find common dominator of all predecessors
- Iterate until fixed point

Dominance Frontiers:

For each block B with multiple predecessors:

- For each predecessor P:
- Walk up dominator tree from P
- Add B to dominance frontier until reaching B's idom

Phi Node Insertion

For each variable V:

1. Collect all blocks where V is defined
2. For each defining block D:
 - For each block F in DF(D):
 - Insert phi node for V in F
 - If F is a new definition site, add to worklist

Variable Renaming

Traverse dominator tree in depth-first order:

1. Process phi nodes in current block
2. Rename variable definitions
3. Update phi operands in successor blocks
4. Recursively process dominated blocks
5. Pop variable versions when leaving block

Usage Example

```
#include "SsaConstruction.hpp"

// Create control flow graph
ControlFlowGraph cfg;
BasicBlockID entry = cfg.createBlock("entry");
BasicBlockID left = cfg.createBlock("left");
BasicBlockID right = cfg.createBlock("right");
BasicBlockID merge = cfg.createBlock("merge");

cfg.setEntryBlock(entry);
cfg.addEdge(entry, left);
cfg.addEdge(entry, right);
cfg.addEdge(left, merge);
cfg.addEdge(right, merge);

// Convert to SSA
SsaConstructor constructor;
auto ssa = constructor.convertToSsa(cfg);

// Validate result
bool valid = SsaUtils::validateSsa(*ssa, cfg);

// Print SSA form
std::string output = SsaUtils::printSsa(*ssa, cfg);
std::cout << output << std::endl;
```

Implementation Details

Complexity

- Dominance computation: $O(n^2)$ worst case, $O(n)$ for reducible graphs
- Phi insertion: $O(n \times m)$ where n = blocks, m = variables
- Variable renaming: $O(n + e)$ where e = edges

Memory Management

- Uses `std::unique_ptr` for ownership
- Efficient adjacency list representation for graphs
- Stack-based variable versioning

Edge Cases

- Irreducible control flow graphs
- Self-loops
- Multiple entry/exit points
- Empty basic blocks

Testing

The SSA construction is tested with 100+ unit tests covering:

- Basic CFG patterns (linear, diamond, loop)
- Dominance computation correctness
- Phi node insertion at merge points

- Variable renaming and versioning
- Complex CFG structures (nested loops, multiple exits)
- Edge cases (empty graphs, self-loops, irreducible CFGs)
- Performance with large graphs

References

1. Cytron et al., "Efficiently Computing Static Single Assignment Form and the Control Dependence Graph" (1991)
2. Appel, "Modern Compiler Implementation in C" (1998)
3. Cooper & Torczon, "Engineering a Compiler" (2011)